Math 128a: Worksheet 1

Student: William Sealy Gosset

September 11, 2025

1 Introduction

What is numerical analysis? Numerical analysis is the intersection of numerical methods and mathematical analysis. Numerical methods are algorithms that approximate solutions to mathematical problems. Analysis is short for analysis of the infinite, a name originating from an eighteenth-century treatise by Leonhard Euler titled *Introductio in analysin infinitorum*. The "infinite" here refers to the infinities studied in the calculus of infinitesimals, a field that today we simply call "calculus." While the term "calculus" can more generally refer to any system of calculation, e.g., propositional calculus, lambda calculus, and Schubert calculus, the calculus you learned in Math 1a and 1b or equivalent was specifically the calculus of infinitesimals, and mathematical analysis is the field that elaborates upon that calculus.

Numerical methods and analysis can interact with each other in a number of ways. Some numerical methods require analysis for justification or proofs of correctness. Some numerical methods are for problems that arise in analysis, such as solving differential equations. The whole framework of numerical methods relies on analytical concepts like approximation and the convergence of sequences.

Programming in Math 128a. Classically, numerical methods were done by hand or with mechanical calculators like slide rules, but since the development of the transistor, mathematicians have come to program their algorithms on modern computers in modern programming languages. In this course, we will use Matrix Laboratory (MATLAB), which is a proprietary language designed specifically for numerical methods. Your license comes with your enrollment in this university. If you are a visiting student, you likely have a license with your home institution.

After this course, you might also find yourself having to use other languages for numerical programming, like Julia, Python (Numpy and PyTorch), R, and Stata. They all differ in maturity, feature richness, interpretation strategy, performance, and licensing, but the skills are generally transferable, and knowing one means that you can learn any other within an afternoon.

Programming with AI. LLMs are powerful tools, but as of 2025, they are still quite fallible. In my personal experience, LLMs often get things wrong if they hallucinate or have outdated knowledge. Even if it has a correct implementation, that implementation might not be the most elegant, clear, or efficient. Relying too heavily on AI for programming tasks will lead to code that is difficult to debug or tune to your specific needs. It will make the learning process harder, and even beyond the learning process, writing code yourself will be faster than prompting the AI and fixing its mistakes.

My guidance is to use AI to give you the general approach or structure for your programs. Read its code and take inspiration from its ideas, but do not copy it over. AI is also helpful for giving you a starting point for usage of libraries, but I strongly suggest following up directly with the documentation if it is available. Lastly, AI is surprisingly effective at test case generation as long as the desired behavior of your program is self-documenting.

A hat problem. We will use any remaining time in this optional section to group up and solve a little puzzle courtesy of Tomasz Schang.

Problem 1.1. You and a friend enter a room and each receive one hat. The hats can be black or white. You each can see the other person's hat, but you cannot see your own. Simultaneously, you both must guess

the color of your own respective hats. You are allowed to agree on a strategy beforehand, but once you enter the room, you are not allowed to signal or communicate anything.

(a) Determine a strategy that guarantees that at least one of you will guess correctly.
(b) Repeat the problem with 3 people and 3 colors.
(c) Repeat the problem with n people and n colors.

2 Calculus

The first two problems that you will encounter in this course are root finding and minimization. These are the problems of finding the solution to $f(x) = 0$ and minimizing f over a region. These problems have important applications in machine learning and computational chemistry. Note: The plurals of minimum, maximum, infimum, supremum, etc. are minima, maxima, infima, suprema, etc.

Problem 2.1. Can you turn a maximization problem into a minimization problem?

Problem 2.2. What theorems from calculus give you preconditions that guarantee the existence of roots and minimizers? Given your precondition for the existence of a root, can you use recursion to approximate the root? How quickly does this algorithm converge? How many significant digits do you get per iteration?

At this point in your mathematical development, you should start paying attention to the conditions in the statements of each theorem and the general intuition used to justify each theorem.

Problem 2.3. The extreme value theorem only applies to continuous functions on a compact domain. Find an example of a bounded continuous function $f : \mathbb{R} \to \mathbb{R}$ without any critical points.

Problem 2.4. The intermediate value theorem only applies to continuous functions on a connected domain. Find a continuous function defined on a disconnected domain that violates the intermediate value theorem. Does the extreme value theorem still hold on this domain?

Problem 2.5. A function $f : \mathbb{R} \to \mathbb{R}$ is periodic if there exists a period L for which $f(x) = f(x + L)$ on all inputs x. What is an example of a periodic function? Can you find and classify the critical points of this function? Can the extreme value theorem and intermediate value theorem be applied to periodic functions?

Problem 2.6. For functions that are continuously differentiable, the mean value theorem follows from applying the intermediate value theorem to the derivative. But the mean value theorem tells you a bit more than this, since it holds even when the derivative isn't continuous. Consider the following function.

$$f(x) = \begin{cases} x^2 \sin \frac{1}{x} & x \neq 0 \\ 0 & x = 0 \end{cases}$$

Is f continuous? Is f differentiable? Is f continuously differentiable? Plot the derivative and convince yourself that it is discontinuous and still satisfies the conclusion of the intermediate value theorem.

3 Taylor Series

Before we write down any equations, it is important to establish the intuition behind Taylor approximations. Many concepts in mathematics have important geometric or physical descriptions that can be obscured when you work too closely with formulae.

Let's say that Alice is driving a car on the highway. Her position at time 0 is known. Let's say we also know her velocity at time 0. This isn't enough to know where she will be in an hour, since her velocity could change drastically during the interim, but if you can somehow guarantee that the velocity doesn't deviate too far from what it was at time 0, you can give a linear estimate of her position. Rigorously, you would be applying the fundamental theorem of calculus and bounding the integrand. This is the first-order Taylor approximation.

Let's say that we know Alice's velocity at time 0, but we no longer guarantee that her velocity stays close to that value. Instead, suppose the know how much her velocity is changing at time 0. In other words, suppose we know Alice's acceleration at time 0. If we additionally suppose that her acceleration doesn't deviate too far from what it was at time 0, we can determine the maximum and minimum possible position of Alice's car by applying the fundamental theorem of calculus twice and bounding the integrand. This gives you a quadratic expression called the second-order Taylor approximation.

In this way, polynomials show up naturally when doing Taylor approximations because they are the result of assuming that position, velocity, acceleration, etc. are constant.

Problem 3.1. Alice is driving down a straight highway. Over the next hour, she holds her speed between 90 and 100 kilometers per hour. Give an upper- and lower-bound on her position at time t.

Problem 3.2. Suppose that Alice's speedometer currently reads 100 kilometers per hour. Over the next hour, she decelerates between 0 and 10 kilometers per hour per hour. Give an upper- and lower-bound on her position at time t.

Problem 3.3. Find the unique polynomial p of order k for which $p(0) = f(0)$, $p'(0) = f'(0)$, etc.

Basic Taylor series you should know and how to manipulate them. If you know some basic Taylor series, you can find Taylor series for more complicated expressions by transforming them.

Problem 3.4. Recall the Maclaurin series for e^x and $1/(1-x)$.

(a) Give a short justification for each series you just recalled.
(b) How might you determine the Maclaurin series for $\sin x$, $\cos x$, $\sinh x$, $\cosh x$, $\log(1+x)$, and $\arctan x$? Make note of the first few terms of the $\sin x$ and $\cos x$ series. They are famously known as the small angle approximations. Check these approximations with some small angles like 0.2 and 0.02. Would these be overestimates or underestimates?
(c) The standard Gaussian is $\exp(-x^2/2)$, up to a normalizing constant. Determine the Maclaurin series for the Gaussian. Use the third-order approximation to estimate

$$\frac{1}{\sqrt{2\pi}} \int_{-1}^{1} e^{-x^2/2}\, dx = \sqrt{\frac{2}{\pi}} \int_{0}^{1} e^{-x^2/2}\, dx.$$

(d) Without additional calculation, how might you expand $1/x$ or $\log x$ around some arbitrary point a?
(e) Compute the first-order Taylor approximation for $\sqrt{1+x}$ about 0. Use rescaling to compute the first-order Taylor approximation of $\sqrt{x}$ about 25. Then approximate $\sqrt{23}$. Would this be an overestimate or an underestimate? What about the second-order approximation?
(f) How might you approximate $(1/80)^{1/40}$? Hint: Is it closer to 0 or 1?

Limitations of Taylor series. Not all functions can be approximated by Taylor series. There is a Wikipedia article on so-called non-analytic smooth functions. The standard example $\exp(-1/x^2)$ flattens so quickly that derivatives are not enough to distinguish it from a constant function. The Fabius function cannot be approximated with a Taylor series anywhere. In this class, you may ignore such functions, but keep in mind that they exist.

4 Asymptotics

When we compare asymptotic expressions, we are describing the relative size of some dependent variables in the limit as some independent variable approaches some limit. Normally, one would need to specify the limit of the independent variable, but often, for certain letters, the limit is understood. Conventionally, n is an integer that goes to $+\infty$, and ϵ or δ is a positive real number that approaches 0. Most other letters have no such convention, like r, t, or x, so their limits have to be specified unless understood from context. It is also possible to have multiple independent variables moving at the same time, in which case you might have to describe the relationship between the variables, i.e., how they compare asymptotically to each other. This is called a regime.

To compare two asymptotic expressions, there are two approaches: We can give a hard bound when the independent variable is restricted to a small neighborhood of its limit or we can take the limit of the relative size of the two dependent variables. In the latter approach, the relative size between two asymptotic expressions should go to 0, ∞, or a constant in between. Note that a neighborhood of a finite number is a small interval around that number, and a neighborhood around infinity is an infinite ray on the number line beginning at a very large number.

There are a number of ways to notate these comparisons. Below is a nonexhaustive list.

$$f(n) \ll g(n) \qquad f(n) \lesssim g(n) \qquad f(n) = O(g(n)) \qquad f(n) = o(g(n))$$
$$f(n) = \Theta(g(n)) \qquad f(n) \sim g(n) \qquad f(n) = \omega(g(n)) \qquad f(n) = \Omega(g(n))$$

Some of these don't even have consistent definitions throughout the literature.

Problem 4.1. Ask your GSI to define some of these, and determine how to compare 1, n, $2n$, $n \log n$, $n \log^2 n$, n^2, $1/n$, e^n, e^{2n}, $e^{-n^2/2}$, and 2^n.

Problem 4.2. What does it mean for $f(n) = o(1)$? What does it mean for $f(n) = g(n) + o(1)$? Show that $f(n) = 1/(1 + o(1))$ implies that $f(n) = 1 + o(1)$.

Problem 4.3. The naked logarithm is used to mean base e, 2, or 10, depending on your field's convention. Explain why the expression $O(\log n)$ is independent of your choice of base. Is $O(b^n)$ independent of your choice of base?

Problem 4.4. The naked logarithm is used to mean base e, 2, or 10, depending on your field's convention. Explain why the expression $O(\log n)$ is independent of your choice of base. Is $O(b^n)$ independent of your choice of base?

Problem 4.5. If there exists a degree k for which $f(n) = O(n^k)$, we say that $f(n) = \text{poly}(n)$. Show that $\text{poly}(n)$ is equivalent to $\exp(O(\log n))$. Is it equivalent to $2^{O(\log n)}$?

5 Error Analysis

In this course, we assume that an algorithm runs for n iterations and express the worst-case error in terms of the runtime. This course's approach to error analysis is only one of many. This worksheet elaborates upon our approach and introduces some alternatives.

Expressing runtime in terms of error. It is possible to flip the roles of error and runtime while describing the efficiency of an algorithm. Suppose that we want to approximate the solution within an interval of radius ϵ. You can describe the required runtime as a function of ϵ. This convention is often used in computer science, where the runtime of an algorithm can depend on many factors beyond just the size of your error bound, such as the size of your data structures or the probability that the error bound is allowed to fail in a probabilistic algorithm.

Problem 5.1. Translate the following rates of convergence into runtime expressions that depend on ϵ.

(a) $O(1/n^2)$
(b) $O(2^{-n})$

Extracting power laws from a sequence. The terms "rate" and "order" can refer to many things throughout the literature. Sometimes, it is used to mean the geometric rate or the power law of a sequence.

Problem 5.2. Compare the expressions $1/n^p$, r^{-n}, $\exp(-n \log R)$, and $\exp(-n^P \log R)$. If a_n takes on each of these forms, how might you use ratios, roots, logarithms, limit infima, and/or limit suprema to extract p, r, R, and P? If two sequences produce the same geometric rate r under your extraction method, are they necessarily asymptotically equivalent?

Error propagation. When you do numerical methods, your final answer isn't the only approximation. Every intermediate number in your computation is an approximation. While less critical than in previous centuries, it is helpful to understand how errors accumulate and propagate throughout a calculation, since this can inform how one should structure a calculation to reduce unnecessary errors.

In as recent as 1991 Gulf War, the Patriot missile defense system failed to intercept an incoming missile battery because of accumulated errors from multiplying by 1/10, which has an imprecise binary representation, rather than dividing by 10, which is a slower operation. You can read more about this in *Bits and Bugs*.

Problem 5.3. When you add two approximations, what is the error of the sum? How might you express this fact in mathematics? How might you use the triangle inequality to prove this fact?

Problem 5.4. Describe how you might use Taylor polynomials to propagate errors across arbitrary functions.

Problem 5.5. In this class, we analyze worst-case error bounds. But if errors come from real-life phenomena, such as instrument precision, we might believe that errors are independent and could cancel each other out. If you add a bunch of independent errors, the central limit theorem tells you that your distribution should look Gaussian regardless of how individual errors are distributed. Instead of looking at worst-case error bounds, one might instead consider error as a multiple of the standard deviation of the error distribution. If you are familiar with probability, ask your GSI what error propagation in quadrature means and describe why we would use absolute error propagation in quadrature for addition and relative error propagation in quadrature for multiplication. Is this an appropriate model for the accumulated error in the 1991 Patriot missile defense system?

6 IEEE 754

The floating-point number format is one of the greatest works of engineering in the history of computing. Let's walk through why it is the way it is.

Problem 6.1. Explain how the floating-point number format is essentially scientific notation. What advantages does a floating-point number format have over a fixed-point number format in storing the precision that we care about? In what circumstances might you prefer a fixed-point number format?

Problem 6.2. Explain the role of the bias in the exponent bits. Explain why we drop the leading bit of the significand and determine whether this would also work for decimal floating-point numbers.

Problem 6.3. The IEEE 754 defines, among other things, the half-precision floating-point (float16) format as the binary floating-point format that uses

I. a single bit for the sign s,
II. five bits for the biased exponent e with bias 15, and
III. ten bits for the mantissa f.

This number format uses sixteen bits in total. Write down the binary encoding of 42 as a float16.

Problem 6.4. How many integers can be represented by float16? What is the largest integer that can be represented by float16? How does this compare to int16?

Endianness. The order in which we write bits down on paper might not match how they are stored on the machine or printed out when visualized, such as in a hex dump. Conventions for byte addressing and byte order were not set in stone during the early history of computing. Today, most computers are little-endian, while most networks are big-endian. This creates a surprising amount of headache for network libraries.

To avoid confusion when dealing with hardware representations of numbers in low-level code, instead of saying left-most bit and right-most bit, you can call them the most-significant bit (MSB) and the least-significant bit (LSB). Regardless of architecture and hardware representation, you should consider the left side as the MSB and the right side as the LSB. This gives the right convention for bit-shift operations outside of some obscure corner cases, since bitwise operations abstract away the hardware representation.

7 Pair Programming

The best way to learn a programming language is to think of something to build in that language and then try to build it. Over the course of writing your program, you will naturally end up researching and learning many of the tools needed for that program. In this part of the worksheet, we provide some basic programming projects to get you started. They can be done on your own, but if you have someone else with you, pair programming is an effective pedagogical tool and channel for knowledge exchange.

Pair programming is the practice of programming as a group of two. There are many ways to pair program, but in this section, we will be using the driver/navigator style of pair programming. If you want to pair program, find a partner. If there aren't an even number of you, some of you can work alone or work in a group of three.

Choose one person to be the driver. This person will be at the keyboard. Only the driver is allowed to code. The driver owns the syntax of the program. The other person will be the navigator. This person will not touch the keyboard. Instead, the navigator will own the structure of the program. The navigator is the one who decides how to split the problem up, how to name functions and variables, and so forth. If there is a third person in the group, that person will be in charge of researching documentation or searching the course material.

While pair programming, the navigator will dictate to the driver what the behavior of the program should be. For instance, the navigator might tell the driver to create a variable named x and initialize it with a 3×3 identity matrix. The driver will figure out how to write that in code. If either of them need external resources, such as documentation or information on the algorithm they are implementing, either person can look up the information. If there is a researcher in the group, the researcher will perform this job.

Throughout this process, the navigator review what the driver writes, and the driver should pay attention to the navigator's overall plan. To this end, both people have to communicate what they are thinking and how they are approaching their work. Either person is allowed to make suggestions to the other person, but the driver makes the final call on the syntax, and the navigator makes the final call on the overall program structure. After 10 minutes, the two people switch roles.

Problem 7.1. Use the Monte-Carlo method to estimate π. In other words, sample a million points within the unit square and calculate the fraction of points that land in the unit disk. Figure out how to extract an approximation of π from this fraction. Here are some follow-up questions.

(a) How might you estimate the volume of the unit ball in $\mathbb{R}^d$?
(b) What is the worst-case error for this algorithm? Why might the average-case error be a lot better? How might you use Hoeffding's inequality, the Chernoff bound, or Cramér's theorem to describe the efficiency of this algorithm more accurately?

Problem 7.2. Remind yourself of how Euler's method works. Figure out how to represent a differential equation generically and perform Euler's method on it.

Problem 7.3. Read up on the Euclidean algorithm and implement it.

Problem 7.4. Read about RPN calculators and implement one on the terminal.

Problem 7.5. Write a program that plots the Julia set.

Problem 7.6. Write a program that takes in a function handle and a starting interval and performs the bisection method or fixed-point iteration. Use it to solve $xe^x = 1$. Check the answer against $W(1)$ in WolframAlpha or Mathematica.